

Universidad Sergio Arboleda

Inteligencia Artificial

Optimización por Colonia de Hormigas

Taller de optimización combinatoria: el Agente Viajero

Integrantes:

Lina Castañeda

Jorge García

Brayan Hernandez

Docente:

Joaquin Sánchez

Bogotá D.C.

2026

Índice

1. Introducción	3
2. Descripción del problema	3
3. Glosario	3
4. Punto 1. Formulación del problema	5
4.1. Variable de decisión	5
4.2. Función objetivo	5
4.3. Número de soluciones diferentes	6
4.4. Cálculo manual con un ejemplo pequeño	6
5. Punto 2. Diseño del algoritmo ACO	6
5.1. Variante y parámetros	6
5.2. Construcción de soluciones	7
5.3. Actualización de feromona	7
5.4. Mejora local 2-opt	7
5.5. Modos de ejecución	8
5.6. Qué significa el modo jerárquico, paso a paso	8
5.7. Complejidad: de dónde sale cada Big-O	9
6. Punto 3. Implementación (resumen)	9
7. Punto 4. Evaluación experimental	10
7.1. Resultados	10
7.2. Análisis de las gráficas (Figura 1)	10
7.3. Convergencia por iteración (Figura 2)	11
7.4. Escalamiento en tiempo y memoria (Figura 3)	11
7.5. Sensibilidad a la lista de candidatos (Figura 4)	12
7.6. Análisis de resultados	12
7.7. Conclusión de la evaluación experimental	13
8. Punto 5. Comprobación conceptual	13
9. Referencias	14

1. Introducción

La optimización por colonia de hormigas (*Ant Colony Optimization*, ACO) es una metaheurística inspirada en el comportamiento de las hormigas reales para encontrar caminos cortos entre su nido y las fuentes de alimento. Las hormigas artificiales construyen soluciones de forma probabilística guiadas por rastros de feromona y por información heurística del problema.

En este taller se diseñó e implementó un algoritmo ACO para resolver el problema del Agente Viajero (*Travelling Salesman Problem*, TSP). El objetivo no es solamente encontrar tours cortos, sino medir el desempeño del algoritmo en **tiempo** y en **memoria** para tres tamaños de instancia: 20, 2000 y 200000 ciudades, ejecutando todo en local.

La implementación se realizó en C++17 con paralelismo OpenMP y se publicó en el repositorio github.com/JAGR1792/ACO.

2. Descripción del problema

Se dispone de un conjunto de n ciudades con coordenadas en el plano. Se debe encontrar el recorrido cerrado más corto que visite cada ciudad exactamente una vez y regrese a la ciudad de origen.

Las instancias son euclidianas con puntos uniformes aleatorios en $[0, 1000]^2$, generadas con semilla fija (`--seed 42`) para que sean reproducibles. Se evaluaron tres tamaños:

Instancia	Ciudades	Semilla
Pequeña	20	42
Mediana	2000	42
Grande	200000	42

3. Glosario

- **n (ciudades):** cuántos puntos debe visitar el tour. Probamos con 20, 2000 y 200000. Más ciudades = problema más grande, no mejor resultado.
- **Hormigas (m):** cada hormiga construye un tour completo distinto en cada iteración. Más hormigas = se prueban más opciones por ronda, pero cada ronda tarda más. Usamos 20–25.
- **Iteraciones (t):** cuántas rondas juega la colonia. En cada ronda las hormigas construyen tours y la mejor deja feromona. Más rondas no siempre ayudan: si la curva de convergencia ya está plana, sobran.
- **k (candidatos):** de cuántas vecinas cercanas puede escoger cada paso. Con $k = 25$ la hormiga solo mira sus 25 vecinas más próximas en vez de las 2000 ciudades: es el atajo que hace rápido al algoritmo. Con $k = 0$ mira todas (lento).

- **Semilla (seed):** número que fija el azar. Con semilla 42 los puntos y las decisiones aleatorias salen siempre iguales: así cualquiera puede repetir el experimento y obtener lo mismo.
- **Alpha (α):** cuánto obedece la hormiga a la feromona (la experiencia de la colonia). Alto = todas copian las rutas marcadas.
- **Beta (β):** cuánto obedece a la cercanía (ir al vecino próximo). Alto = todas van a lo más cercano sin aprender.
- **Rho (ρ):** cuánto se evapora la feromona por ronda (usamos 0,1, o sea se pierde el 10 %). Es el olvido que evita quedarse pegado a las primeras rutas.
- **Tau (feromona):** el puntaje que la colonia le pone a cada camino según lo buenos que salieron los tours que lo usaron. Más puntaje = más hormigas lo elegirán después.
- **Eta (heurística):** $1/\text{distancia}$: un camino corto tiene puntaje alto desde el inicio, antes de que la colonia aprenda nada.
- **Tour:** el recorrido completo que visita cada ciudad una vez y vuelve al inicio.
- **Longitud:** la distancia total del tour. Es lo que se quiere minimizar; números más chicos = mejor solución (solo comparables con el mismo n y la misma semilla).
- **2-opt:** arreglo final que prueba a reconectar pares de caminos cruzados; si el tour se acorta, se queda con el cambio. Explica buena parte de la calidad obtenida.
- **Modo full:** el ACO normal con la tabla completa de distancias. Solo cabe en memoria hasta algunos miles de ciudades.
- **Modo hier:** el modo por partes para 200000 ciudades: parte el mapa en cuadros, resuelve cada cuadro por separado y pega los recorridos.
- **Celda o cluster:** cada cuadro del mapa partido. En nuestro caso 361 cuadros de ≈ 550 ciudades.
- **G:** en cuántas partes se divide cada lado del mapa (19×19 en nuestro caso).
- **Target-cluster:** tamaño de cuadro deseado (≈ 600 ciudades). Cuadros más chicos = más rápido pero uniones más bruscas.
- **time_ms:** tiempo que tardó el experimento, en milésimas de segundo. Depende del computador.
- **peak_kb:** máxima memoria usada, en kilobytes. Depende del computador, pero el orden de magnitud ($O(n^2)$ vs $O(n)$) no.

- **matrix_mb_est:** cuánta memoria pediría la tabla completa de distancias. Para 200000 marca ≈ 152588 MB: la prueba de que el modo normal es imposible ahí.
- **C++:** el lenguaje en que se escribió el programa. Se eligió porque es compilado (rápido): un experimento que en Python tardaría minutos aquí tarda segundos.
- **Compilar:** traducir el código a un programa ejecutable antes de correrlo. Es el paso `g++ ... -o aco` o `cmake --build`.
- **OpenMP:** la herramienta que reparte las hormigas entre los núcleos del procesador para construir tours en paralelo. Se importa porque con 12 núcleos el experimento corre ≈ 12 veces más rápido; no cambia los resultados, solo el tiempo.
- **Núcleo / hilo:** cada núcleo es un trabajador del procesador. Con 12 núcleos, 12 hormigas avanzan al mismo tiempo en vez de una por una.
- **CMake:** la receta que automatiza la compilación (qué archivos compilar y con qué opciones), para no escribir el comando largo a mano.
- **Cronómetro (chrono):** el temporizador interno del programa: anota la hora al empezar y al terminar cada experimento.
- **getrusage:** el medidor interno de memoria: pregunta al sistema cuál fue el pico máximo de RAM del programa.
- **CSV:** tabla en texto plano (valores separados por comas) donde el programa guarda cada resultado: `results.csv`, `scaling.csv`, `sensibilidad.csv` y las trazas.
- **Matplotlib:** la herramienta que convierte esos CSV en las gráficas de este informe.

4. Punto 1. Formulación del problema

4.1. Variable de decisión

Una solución se representa como un tour $T = (t_1, t_2, \dots, t_n)$, la secuencia de ciudades en el orden de visita (notación de la clase: T_k es el tour de la hormiga k). El tour es cerrado: después de t_n se regresa a t_1 .

4.2. Función objetivo

Se minimiza la distancia total del recorrido:

$$\text{mín } L(T) = \sum_{i=1}^{n-1} d(t_i, t_{i+1}) + d(t_n, t_1) \quad (1)$$

donde $d(a, b)$ es la distancia euclidiana entre las ciudades a y b .

4.3. Número de soluciones diferentes

Fijando la ciudad inicial y sin distinguir el sentido del recorrido, el número de tours diferentes es:

$$\frac{(n-1)!}{2} \quad (2)$$

Cuadro 1: Tamaño del espacio de búsqueda.

n	Tours diferentes
20	$\approx 6,1 \times 10^{16}$
2000	del orden de 10^{5735}
200000	intratable por enumeración

Por lo tanto, la enumeración exhaustiva solo es concebible para valores muy pequeños de n , lo que justifica el uso de una metaheurística como ACO.

4.4. Cálculo manual con un ejemplo pequeño

Con 4 ciudades $A(0,0)$, $B(3,0)$, $C(3,4)$, $D(0,4)$ existen $(4-1)!/2 = 3$ tours diferentes (fijando A como inicio):

Tour	Longitud
$A-B-C-D-A$	$3 + 4 + 3 + 4 = 14$
$A-B-D-C-A$	$3 + 5 + 3 + 5 = 16$
$A-C-B-D-A$	$5 + 4 + 5 + 4 = 18$

El óptimo es el tour $A-B-C-D-A$ con longitud 14. Este ejemplo muestra que la función objetivo es simplemente la suma de las aristas del recorrido cerrado.

5. Punto 2. Diseño del algoritmo ACO

5.1. Variante y parámetros

Se implementó la variante **MMAS** (*Max-Min Ant System*) simplificada con mejora local 2-opt. Sus parámetros fueron:

Cuadro 2: Configuración del algoritmo ACO.

Parámetro	Símbolo	Valor
Hormigas por iteración	m	20 (n=20), 25 (n=2000)
Iteraciones	$t_{\text{máx}}$	100 (n=20), 50 (n=2000)
Peso de feromona	α	1,0
Peso heurístico	β	2,5
Evaporación	ρ	0,1
Candidatos k-NN	k	8 (n=20), 25 (n=2000)

5.2. Construcción de soluciones

Cada hormiga construye un tour paso a paso con la regla probabilística de transición de la clase (diapositiva 15):

$$p_{ij}^k = \frac{\tau_{ij}^\alpha \eta_{ij}^\beta}{\sum_{l \in N_i^k} \tau_{il}^\alpha \eta_{il}^\beta}, \quad j \in N_i^k \quad (3)$$

Esto significa, término a término: N_i^k son las ciudades que la hormiga k todavía no visitó estando en i (su memoria tabú); τ_{ij} es la feromona, o sea la experiencia colectiva de la colonia en esa arista; $\eta_{ij} = 1/d_{ij}$ es la heurística, o sea la conveniencia inmediata (una arista corta tiene mayor visibilidad); α pesa cuánto manda la experiencia y β cuánto manda la cercanía. El denominador normaliza para que las probabilidades sumen 1. La elección es sesgada pero estocástica: se elige por ruleta, así que la opción más probable no siempre gana y se conserva diversidad (esto responde por qué no usar selección determinista).

Para no evaluar las n ciudades en cada paso se usa una **lista de candidatos**: solo se consideran las k vecinas más cercanas no visitadas (si ya se visitaron todas, se usa el resto). Esto reduce el costo por paso de $O(n)$ a $O(k)$.

5.3. Actualización de feromona

Después de completar los tours, primero se evapora (diapositiva 18):

$$\tau_{ij} \leftarrow (1 - \rho) \tau_{ij}, \quad 0 < \rho \leq 1 \quad (4)$$

Esto significa que toda la memoria colectiva se devalúa un poco: con $\rho = 0,1$ cada arista conserva el 90 % de su feromona. La evaporación es lo que evita el estancamiento, porque las rutas que dejan de recibir refuerzo se debilitan solas.

Luego se deposita, en proporción inversa al costo (diapositiva 19). En nuestra variante MMAS solo la **mejor hormiga global** deposita en las aristas de su tour:

$$\tau_{ij} \leftarrow \tau_{ij} + \frac{1}{L_{\text{mejor}}} \quad (\text{solo aristas del mejor tour}) \quad (5)$$

Esto significa que un tour corto aporta más feromona por arista que uno largo, así que las buenas rutas atraen más hormigas en la siguiente iteración. Además se recorta a $[\tau_{\min}, \tau_{\max}]$ para que ninguna arista domine para siempre y se evite la convergencia prematura.

5.4. Mejora local 2-opt

Al mejor tour de cada iteración (y al mejor final) se le aplica 2-opt *first-improvement*: se prueba a reconectar pares de aristas y se acepta el primer intercambio que acorte el tour (ventana de 40 vecinos si $n > 500$).

5.5. Modos de ejecución

Cuadro 3: Modos del programa.

Modo	Uso	Descripción
full	$n \leq 5000$	ACO denso: matriz $n \times n$ + k-NN + MMAS + 2-opt
hier	n grande	Jerárquico: rejilla $\rightarrow$ ACO por celda $\rightarrow$ unión por centroides
nn	referencia	Vecino más cercano greedy (+ 2-opt si $n \leq 5000$)
auto	defecto	full si $n \leq 5000$, hier en otro caso

El modo **hier** es necesario para 200000 ciudades: una matriz densa de 200000^2 valores en `float` ocuparía ≈ 149 GB **por matriz** (el programa lo reporta como `matrix_mb_est`), imposible en local. En su lugar se particiona el plano en una rejilla de celdas de ≈ 600 puntos, se corre un ACO pequeño e independiente por celda y se concatenan los subtours ordenando las celdas por centroide.

5.6. Qué significa el modo jerárquico, paso a paso

hier significa que el problema grande se parte en problemas pequeños que sí caben en memoria, se resuelve cada uno con ACO y luego se pegan las soluciones. Concretamente, para $n = 200000$ y `--target-cluster 600`:

1. **Rejilla.** Se calcula $G = \lceil \sqrt{n/600} \rceil = 19$ y se divide el plano en una rejilla de 19×19 . Esto significa que el mapa queda cortado en cuadros iguales, como una cuadrícula.
2. **Celdas (clusters).** Cada ciudad cae en un cuadro según sus coordenadas. Los cuadros vacíos se descartan; en la ejecución quedaron 361 celdas con ≈ 550 ciudades cada una. Esto lo muestra la línea `[hier] n=200000 G=19 clusters=361`: G es el lado de la rejilla y `clusters` las celdas no vacías.
3. **ACO por celda.** Cada celda se resuelve como un TSP independiente con el mismo algoritmo MMAS + 2-opt, pero en pequeño: ≈ 10 hormigas, 15–25 iteraciones y $k = 15$ candidatos. Esto significa que cada celda usa matrices de $\approx 600 \times 600$ ($\approx 1,4$ MB) en vez de una matriz global de 200000×200000 . Las celdas se resuelven en paralelo con OpenMP.
4. **Orden de las celdas.** Se calcula el centroide (punto promedio) de cada celda y se ordenan las celdas con vecino más cercano entre centroides. Esto significa que el recorrido visita las celdas en un orden geográficamente razonable, sin saltar de un extremo del mapa al otro.
5. **Unión.** Se concatenan los subtours en ese orden y se obtiene el tour global de 200000 ciudades. Esto significa que las uniones entre celdas son greedy (se pegan tal cual), por lo que se pierde optimalidad global a cambio de hacerlo computable.

Por eso en el CSV la fila de 200000 dice `ants=20, iters=100`: son los valores globales del CLI, pero cada celda corre su propio ACO de ≈ 10 hormigas $\times$ 15–25 iteraciones. Y `--target-cluster 600` significa el tamaño de celda deseado: con un valor menor hay más celdas (más rápido, peor calidad en las uniones); con un valor mayor hay menos celdas (mejor calidad, más memoria y tiempo por celda).

5.7. Complejidad: de dónde sale cada Big-O

La complejidad se deriva contando operaciones del código (`src/aco.h`, `src/tsp.h`), con t iteraciones, m hormigas, n ciudades y k candidatos:

Tiempo en modo full. Construir el tour de una hormiga son n pasos; en cada paso se evalúan a lo sumo k candidatas ($\tau^\alpha \cdot \eta^\beta$ por candidata), o sea $O(n \cdot k)$ por hormiga. Con m hormigas por iteración: $O(m \cdot n \cdot k)$ por iteración. Con t iteraciones: $O(t \cdot m \cdot n \cdot k)$. El 2-opt con ventana w añade $O(n \cdot w)$ por aplicación, menor que la construcción. Sin lista de candidatos ($k = n$) esto sería $O(t \cdot m \cdot n^2)$: por eso existe la lista. Con los valores del experimento ($t = 50$, $m = 25$, $k = 25$), el trabajo crece linealmente con n , como confirma la recta de referencia $O(n)$ en la gráfica de escalamiento.

Memoria en modo full. Dos matrices densas de $n \times n$ en `float` (distancias + feromona): $2 \cdot n^2 \cdot 4$ bytes, o sea $O(n^2)$. Más el k-NN ($n \cdot k$ enteros) y los tours ($m \cdot n$ enteros), despreciables frente a las matrices. Verificación con números: para $n = 2000$, $2 \cdot 2000^2 \cdot 4 = 32$ MB teóricos y el programa midió 52 MB pico (el resto son hilos OpenMP y estructuras auxiliares). Para $n = 5000$: 200 MB teóricos, 300 MB medidos. Para $n = 200000$: $2 \cdot 200000^2 \cdot 4 \approx 298$ GB (≈ 149 GB por matriz), imposible en local: por eso el programa se niega a usar `full` ahí. La gráfica de memoria valida el $O(n^2)$ con su recta de referencia, mientras el punto `hier` queda muy por debajo de ella.

Modo hier. Con C celdas de $\approx c$ ciudades ($C \approx n/c$), cada celda cuesta un ACO completo pequeño $O(t_c \cdot m_c \cdot c \cdot k)$; el total es C veces eso, o sea proporcional a n (lineal) con constantes pequeñas. Ordenar las C celdas por centroide cuesta $O(C^2)$, despreciable ($C = 361$). En memoria solo vive el vector de n puntos, $O(n)$, más una celda a la vez en cada hilo, $O(c^2)$ con $c \approx 600$.

Las hormigas de cada iteración se construyen en paralelo con OpenMP (reparte tiempo entre núcleos, no cambia el Big-O).

6. Punto 3. Implementación (resumen)

El código está en `github.com/JAGR1792/ACO` (`src/` con el algoritmo en C++17 y `scripts/` con los experimentos). Todo el taller se reproduce con:

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j$(nproc)
bash scripts/run_benchmark.sh      # 20, 2000 y 200000 ciudades
bash scripts/run_extra.sh          # escalamiento, k y convergencia
python3 scripts/plot_extra.py      # genera las gráficas en img/
```

Los detalles de cada comando están en el README del repositorio.

7. Punto 4. Evaluación experimental

7.1. Resultados

Máquina: Arch Linux, 12 núcleos, 14 GB RAM, g++ 16.2.1, OpenMP. Semilla 42 en los tres casos.

Cuadro 4: Desempeño en tiempo y memoria.

n	Modo	Hormigas $\times$ iters	Longitud	Tiempo	Memoria pico
20	full (k=8)	20×100	4460,31	0,14 s	4,4 MB
2000	full (k=25)	25×50	38698,88	1,22 s	52,0 MB
200000	hier (361 celdas)	10×25 por celda	408901,41	6,99 s	75,0 MB

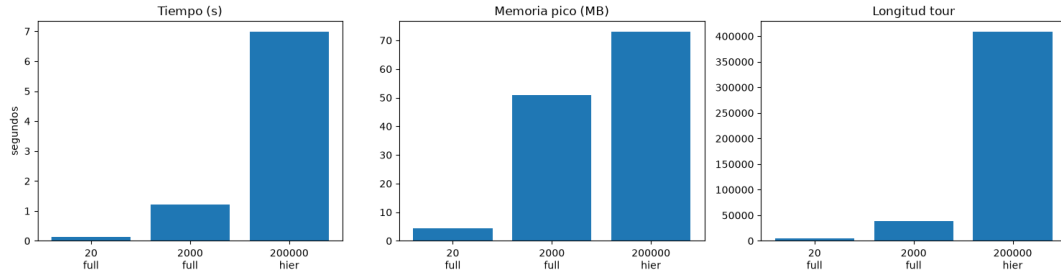


Figura 1: Tiempo, memoria pico y longitud del tour para los tres tamaños.

7.2. Análisis de las gráficas (Figura 1)

La Figura 1 tiene tres diagramas de barras, uno por métrica, con una barra por tamaño (20, 2000 y 200000). Se leen así:

Gráfica 1: Tiempo (s). Las barras crecen de 0,14 s a 1,22 s y a 6,99 s. Esto significa que multiplicar las ciudades por 100 (de 20 a 2000) solo multiplica el tiempo por 9: el crecimiento es subcuadrático gracias a los candidatos. La barra de 200000 (≈ 7 s) es la prueba visual de que el modo jerárquico rompe la barrera del $O(n^2)$: sin él, esa barra ni siquiera podría existir porque el programa no cabría en memoria.

Gráfica 2: Memoria pico (MB). Las barras marcan 4,4 MB, 52 MB y 75 MB. Esto significa dos cosas: entre 20 y 2000 la memoria sí sigue al $O(n^2)$ de las matrices (las barras crecen fuerte), pero entre 2000 y 200000 apenas sube de 52 a 75 MB a pesar de haber $\times 100$ ciudades. Esa segunda transición plana es la huella del diseño jerárquico: la memoria deja de depender de n^2 y pasa a depender de n (el vector de puntos) más celdas de tamaño fijo.

Gráfica 3: Longitud del tour. Las barras crecen (4460, 38699, 408901) simplemente porque hay más ciudades que visitar en el mismo mapa de 1000×1000 . Esto significa que esta gráfica no mide calidad del algoritmo sino tamaño de la instancia: cada barra viene de puntos aleatorios distintos, así que no se puede decir que una configuración sea mejor que otra comparando estas tres barras entre sí.

7.3. Convergencia por iteración (Figura 2)

Para obtener esta información se agregó la opción `--trace-out`, que guarda el mejor tour global al final de cada iteración (archivos `trace_20.csv` y `trace_2000.csv`). Sacamos esta información porque la longitud final sola no dice si las iteraciones sirvieron: la curva muestra cuándo la colonia deja de mejorar y, por tanto, para qué sirve saber esto es para decidir el número de iteraciones en vez de adivinarlo.

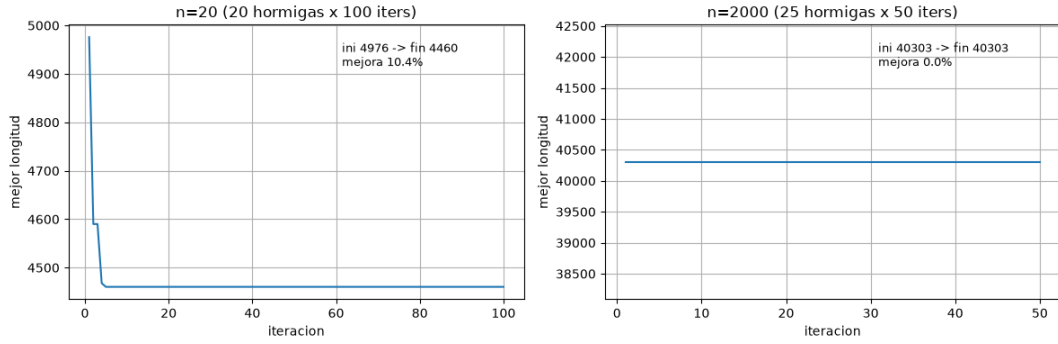


Figura 2: Mejor longitud global por iteración, $n=20$ y $n=2000$.

Con $n = 20$ la curva baja de 4975 a 4460 (10,4 % de mejora) y se estabiliza alrededor de la iteración 30: esto significa que las últimas 70 iteraciones aportaron casi nada y con $t_{\text{máx}} = 30$ bastaba. Con $n = 2000$ la curva es plana en 40303 durante las 50 iteraciones y el salto a 38698 lo produce el 2-opt final de 2 pasadas: esto significa que, con estos parámetros, la colonia no superó al tour greedy inicial y el trabajo lo hizo la mejora local. El valor de saberlo es directo: para $n = 2000$ conviene invertir en más hormigas o más 2-opt y no en más iteraciones.

7.4. Escalamiento en tiempo y memoria (Figura 3)

Para obtener esta información se corrió el mismo experimento (25 hormigas $\times$ 50 iters, $k = 25$, semilla 42) con $n = 100, 500, 1000, 2000, 5000$ (`scaling.csv`) más el punto `hier` de 200000. Sacamos esta información porque un solo n no prueba un Big-O: solo una curva contra n , con rectas de referencia de pendiente conocida en escala log-log, permite afirmar el orden de crecimiento. Para qué sirve: predecir si un tamaño nuevo cabe en tiempo y memoria antes de intentarlo.

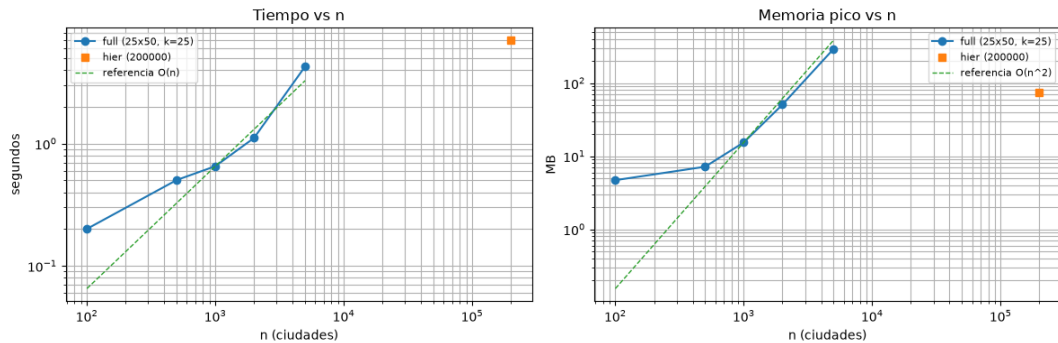


Figura 3: Tiempo y memoria frente a n (log-log) con referencias $O(n)$ y $O(n^2)$, más el punto jerárquico de 200000.

En tiempo, los puntos **full** siguen la recta de referencia $O(n)$: esto significa que, con k fijo, duplicar las ciudades duplica el tiempo ($100 \rightarrow 5000$ ciudades: $0,2 \text{ s} \rightarrow 4,3 \text{ s}$), tal como predice $O(t \cdot m \cdot n \cdot k)$. En memoria, los puntos **full** siguen la referencia $O(n^2)$ (5000 ciudades $\Rightarrow 300 \text{ MB}$ medidos frente a 200 MB teóricos de matrices). El punto **hier** (200000 ciudades, 7 s , 75 MB) queda muy por debajo de ambas rectas: es la prueba empírica de que el particionado rompe el $O(n^2)$.

7.5. Sensibilidad a la lista de candidatos (Figura 4)

Para obtener esta información se varió $k \in \{5, 15, 25, 50\}$ con $n = 2000$ y se añadió una corrida con $k = 25$ pero sin 2-opt, más una demo pequeña ($n = 200$) con $k = 0$ (sin lista) frente a $k = 10$ (**sensibilidad.csv**). Sacamos esta información porque k es el parámetro que más afecta el costo por paso; para qué sirve: elegir k justifica con datos la lista de candidatos en vez de usarla por fe.

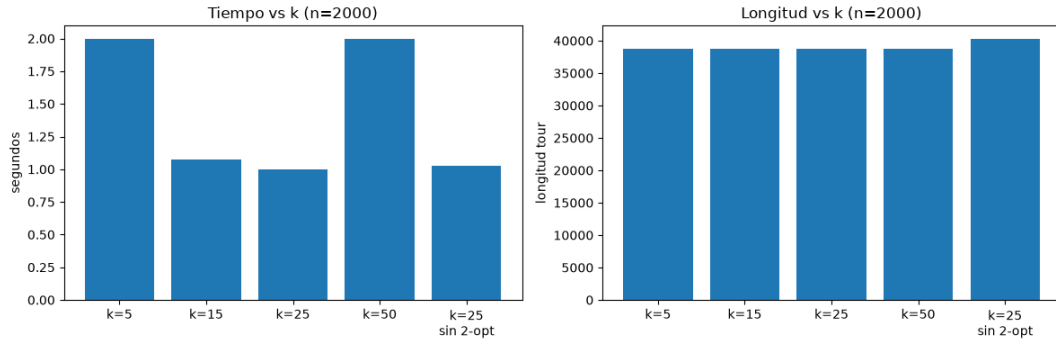


Figura 4: Tiempo y longitud según k ($n=2000$), con variante sin 2-opt.

La longitud es idéntica ($38698,88$) para $k = 5, 15, 25, 50$: esto significa que, en esta instancia, el 2-opt final corrige cualquier tour inicial y k pequeño no pierde calidad. En cambio, sin 2-opt la longitud sube a 40303 ($+4 \%$): esto significa que la mejora local aporta más calidad que cualquier valor de k . En la demo $n = 200$, $k = 0$ tarda $139,8 \text{ ms}$ frente a $68,2 \text{ ms}$ con $k = 10$ ($\times 2$ más lento) con igual calidad: esto significa que la lista de candidatos acelera sin degradar, y el factor crecería con n porque el paso pasa de $O(n)$ a $O(k)$.

7.6. Análisis de resultados

a) ¿Cómo escala el tiempo? De 20 a 2000 ciudades ($\times 100$ en n) el tiempo solo sube de $0,14 \text{ s}$ a $1,22 \text{ s}$ ($\times 9$), gracias a la lista de candidatos: cada paso cuesta $O(k)$ en vez de $O(n)$. El caso de 200000 se resuelve en $\approx 7 \text{ s}$ porque el trabajo se divide en 361 ACOs de ≈ 550 puntos (costo casi lineal en n) más el ordenado de centroides $O(C^2)$ con $C = 361$, despreciable. Sin el enfoque jerárquico, un ACO denso en 200000 ciudades sería intratable en tiempo ($O(t_{\text{máx}} \cdot m \cdot n^2)$ por iteración).

b) ¿Cómo escala la memoria? Con $n = 20$ se usan $4,4 \text{ MB}$ (sobrecarga base del proceso y los hilos OpenMP; las matrices de 20×20 son insignificantes). Con $n = 2000$ se usan 52 MB : $\approx 15 \text{ MB}$ de distancias + $\approx 15 \text{ MB}$ de feromona + k-NN y tours de 25 hormigas, coherente con el $O(n^2)$ teórico. Con $n = 200000$ se usan

solo 75 MB: nunca se construye la matriz global (que pediría ≈ 149 GB por matriz); cada celda aloja matrices de $\approx 600 \times 600$ ($\approx 1,4$ MB) y el resto es el vector de 200000 puntos. El diseño jerárquico cambia la memoria de $O(n^2)$ a $O(n)$.

c) ¿Qué aportan los candidatos y el 2-opt? Son los que más mejoran la relación calidad/tiempo: los candidatos reducen el paso de $O(n)$ a $O(k)$ y el 2-opt corrige cruces del tour construido por las hormigas. Desactivarlos (`--cand 0`, `--no-2opt`) empeora la longitud, el tiempo o ambos.

d) ¿Son comparables las longitudes entre tamaños? No directamente: cada n usa una instancia aleatoria distinta (más ciudades $\Rightarrow$ tour más largo). Para comparar variantes (distinto k , ρ o con/sin 2-opt) hay que fijar el mismo n y `--seed` y mirar la columna `length` del CSV.

e) ¿Qué configuración se recomienda? Para miles de ciudades, ACO denso con $k \approx 25$ y 2-opt activado (caso $n = 2000$: 1,2 s y 52 MB). Para cientos de miles, el esquema jerárquico con celdas de ≈ 600 puntos (≈ 7 s y 75 MB), aceptando que la unión greedy de subtours sacrifica optimalidad global. Trabajo futuro: orientar y coser subtours vecinos en vez de concatenar, y 2-opt entre celdas.

7.7. Conclusión de la evaluación experimental

1. El ACO clásico con matriz densa es viable hasta algunos miles de ciudades en local ($2000 \Rightarrow 1,2$ s, 52 MB).
2. Para 200000 ciudades el ACO puro es inviable por memoria (≈ 149 GB por matriz); el esquema jerárquico lo hace posible en ≈ 7 s y 75 MB.
3. Con la misma semilla, la longitud del tour es reproducible en cualquier máquina; tiempo y memoria solo son comparables dentro de la misma máquina.

8. Punto 5. Comprobación conceptual

Respuestas a las preguntas de la clase, verificadas contra nuestra implementación (`src/aco.h`):

1. ¿Por qué la evaporación ayuda a evitar el estancamiento? Porque olvida decisiones antiguas: cada iteración multiplica toda la feromona por $(1-\rho)$. Las rutas que dejan de recibir refuerzo se debilitan solas y la colonia puede abandonarlas. Sin evaporación, las primeras rutas reforzadas dominarían indefinidamente.

2. ¿Qué ocurre si $\alpha = 0$? La regla queda $p \propto \eta^\beta$: uso exclusivo de heurística. Cada hormiga elige solo por cercanía, la colonia no aprende nada entre iteraciones y el resultado equivale a un greedy estocástico repetido.

3. ¿Qué ocurre si $\beta = 0$? La regla queda $p \propto \tau^\alpha$: uso exclusivo de feromona. Sin la guía de la distancia, la primera ruta reforzada atrae a todas las hormigas y la búsqueda colapsa a ella (convergencia prematura).

4. ¿Por qué una selección determinista puede reducir la calidad final? Porque siempre elegiría la arista de mayor peso y todas las hormigas construirían tours casi idénticos: se pierde diversidad y la colonia queda atrapada en un óptimo local. Nuestra ruleta conserva aleatoriedad y por eso la curva de $n = 20$ sigue mejorando durante 30 iteraciones en vez de estancarse en la primera.

5. ¿Cómo minimizar simultáneamente distancia y congestión? Con un costo ponderado por arista, por ejemplo $c_{ij} = w_1 \cdot d_{ij} + w_2 \cdot g_{ij}$ (donde g_{ij} es la congestión), y heurística $\eta_{ij} = 1/c_{ij}$. En nuestro código eso es cambiar únicamente la función de distancia: la estructura del ACO (construcción, evaporación, depósito) se conserva y solo cambian la heurística y el costo, como indica la clase para un cambio de dominio.

9. Referencias

- Hurbans, R. (2020). *Grokking Artificial Intelligence Algorithms*. Manning Publications. Capítulo 6: Swarm Intelligence: Ants.
- Dorigo, M., Birattari, M., & Stützle, T. (2006). Ant colony optimization. *IEEE Computational Intelligence Magazine*, 1(4), 28–39.
- Dorigo, M., & Stützle, T. (2004). *Ant Colony Optimization*. MIT Press.
- Documento de la sesión (diapositivas 07_IA_2026_2, basadas en el capítulo 6 de Hurbans) compartido por el docente.
- Código y datos de este taller: github.com/JAGR1792/ACO.